

Fundamental of Java Multi-threading

This article explains what threads are in Java. It elaborates on their uses and on how to implement them, while discussing their life cycle.



In a multi-tasking environment, having multiple threads helps the programmer to create efficient programs. In the context of Java, thread means two different things:

- An instance of *java.lang.Thread*
- A thread of execution

An instance of a thread is just an object like any other object in Java. It has variables and methods, and lives and dies on the heap, but a thread of execution is an individual process (a lightweight process) that has its own stack. In Java, there is one thread per stack or vice versa. Even if you don't create any thread in your program, there is a thread running in your program, by default, which is called the Main thread. The *main()* function of Java or the starting point of a Java program runs in the Main thread, or we can say that the Main thread controls the execution of the *main()* function of a Java program.

Why we need threads

In Java, a thread is required to fulfil the purpose of multi-tasking. When we have to perform multiple tasks simultaneously, we need multi-threading. There are two types of threads in Java:

- **User thread:** This is created by the Java programmer or user. Once all the user threads are complete, the JVM will shut down regardless of the state of any daemon threads.
- **Daemon thread:** This is created by the operating system. The JVM doesn't care about the completion of the daemon thread. The best example of a daemon thread is the garbage collector. The Java garbage collector is started by the JVM once any program completes its execution.

Thread implementation in Java

There are two ways to implement threads in Java.

Extending the thread class: We can create a thread